

MODULE 18

Pretraining & Transfer Learning in Gen AI

Topics Covered

- Lecture 1 — Pre-Trained Models
- Lecture 2 — Pre-Training for Gen AI Apps in Industry
- Lecture 3 — Transfer Learning in Neural Networks
- Lecture 4 — How Transfer Learning is Performed in Industry

Lecture 1: Pre-Trained Models

A pre-trained model is a neural network already trained on a large dataset for a general task. Instead of training from scratch — requiring massive data, compute, and months of time — you start with a model that already understands language, images, or audio, and adapt it to your specific need.

The textbook analogy

Training from scratch = writing a textbook from zero.
Using a pre-trained model = starting with an expert's textbook and annotating the relevant chapters.
You get years of encoded knowledge instantly — you only add what is specific to your problem.

Why Pre-Trained Models Matter

- GPT-4 cost ~\$100M and months of GPU time to pre-train — no individual or small company can replicate that.
- Pre-trained models encode rich general knowledge: grammar, world facts, visual patterns, reasoning chains.
- Fine-tuning on your specific task takes hours on a single GPU — not months on thousands.
- With only 100-1000 labelled examples, fine-tuning a pre-trained model outperforms training from scratch on 100K samples.

Landscape of Pre-Trained Models

Domain	Key Models Organisation
Text (LLMs)	GPT-4, GPT-3.5 (OpenAI) Claude (Anthropic) Gemini (Google) LLaMA 3 (Meta) Mistral (Mistral AI)
Text (encoders)	BERT, RoBERTa, DistilBERT ALBERT, DeBERTa Used for classification, NER, search
Images	ResNet, VGG, EfficientNet (CNN) ViT - Vision Transformer CLIP (OpenAI)
Image generation	Stable Diffusion (StabilityAI) DALL-E 3 (OpenAI) Midjourney
Audio / Speech	Whisper (OpenAI) WaveNet (DeepMind) Wav2Vec 2.0 (Meta)
Multimodal	GPT-4V, Gemini, LLaVA, CLIP — handle text + image together
Code	GitHub Copilot (Codex) CodeLLaMA (Meta) StarCoder Gemini Code

Model Hubs

Hub	What You Find
Hugging Face (huggingface.co)	500K+ models, datasets, demo Spaces. The GitHub of AI.
TensorFlow Hub (tfhub.dev)	TF-native models: image, text, audio, video
PyTorch Hub (pytorch.org/hub)	Pretrained PyTorch models: ResNet, BERT, GPT-2
Ollama (ollama.com)	Run LLMs locally: LLaMA 3, Mistral, Gemma, Phi-3

Hub	What You Find
OpenAI API	GPT-4, embeddings (text-embedding-3), Whisper via REST API
<pre># Using a pre-trained model from Hugging Face in 3 lines from transformers import pipeline # Sentiment analysis classifier = pipeline('sentiment-analysis') print(classifier('I love building AI applications!')) # [{'label': 'POSITIVE', 'score': 0.9998}] # Text generation gen = pipeline('text-generation', model='gpt2') print(gen('The future of AI is', max_length=50)[0]['generated_text']) # Zero-shot classification - no fine-tuning needed zs = pipeline('zero-shot-classification') print(zs('I love playing football', candidate_labels=['sports','politics','tech']))</pre>	

Lecture 2: Pre-Training for Gen AI Apps in Industry

What is Pre-Training?

Pre-training is the initial phase where a model learns from a massive general corpus using a self-supervised objective — no human labels required. The model discovers structure, patterns, and knowledge from raw data alone.

Model Type	Pre-Training Objective Dataset
GPT / LLaMA (decoder)	Next-token prediction: given 'The cat sat on', predict 'the'. Trained on trillions of tokens from web, books, code.
BERT (encoder)	Masked Language Modelling: predict [MASK] in 'The cat [MASK] on the mat'. Plus Next Sentence Prediction.
T5 / BART (enc-dec)	Text-to-text: frame every task as input-output text pair. Corruption + reconstruction objective.
CLIP (multimodal)	Contrastive: match image to its correct caption from a batch of 32K image-text pairs.
Stable Diffusion	Denosing diffusion: learn to remove noise from images step by step.
Whisper (audio)	Predict transcript from audio spectrogram. Trained on 680K hours of labelled audio.

Pre-Training Scale in Industry

Model	Parameters Tokens Trained Compute
GPT-3	175B params 300B tokens ~\$4.6M
LLaMA 3 70B	70B params 15T tokens Meta cluster
GPT-4	~1.8T (MoE) Undisclosed ~\$100M
Gemini Ultra	Undisclosed Undisclosed Google TPU pods
BERT-base	110M params 3.3B tokens 4 Cloud TPUs, 4 days

Why companies can't pre-train their own LLM

Pre-training GPT-4 scale requires 10,000+ A100 GPUs running for months.
 The cost, infrastructure, and dataset curation are beyond almost every organisation.
 Industry solution: use an existing pre-trained foundation model and fine-tune it.

Pre-Training Data Pipeline (Industry)

Stage	Details
Data collection	Common Crawl (web), Books, Wikipedia, GitHub, arXiv, StackOverflow, multilingual sources
Deduplication	Remove near-duplicate documents — MinHash, exact-match hashing
Quality filtering	Perplexity-based filtering, language ID, toxic content removal, PII scrubbing
Tokenisation	Train a BPE (Byte Pair Encoding) or SentencePiece tokeniser on the corpus
Training infra	FSDP / DeepSpeed / Megatron-LM for distributed training across thousands of GPUs
Checkpointing	Save model every N steps — allows resuming after hardware failure

```

# Tokenisation example — how text becomes numbers for an LLM
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('meta-llama/Meta-Llama-3-8B')

text = 'Pre-training teaches the model general language understanding.'
tokens = tokenizer(text, return_tensors='pt')

print('Input IDs:', tokens['input_ids'])
print('Token count:', tokens['input_ids'].shape[1])
print('Decoded:', tokenizer.decode(tokens['input_ids'][0]))

# Vocabulary size of LLaMA 3
print('Vocab size:', tokenizer.vocab_size) # 128,256

```

Lecture 3: Transfer Learning in Neural Networks

Transfer learning reuses knowledge from a pre-trained model to solve a new, related task. Instead of random weight initialisation, you start with weights that already encode useful representations — dramatically reducing data and compute requirements.

Human analogy

A doctor learning radiology already knows anatomy, biology, and how to read medical charts. They transfer that knowledge — they only need to learn the specifics of interpreting X-rays. A complete beginner would need years more training to reach the same level.

How Transfer Learning Works

Step	Action
1. Select base model	Choose a pre-trained model whose domain matches your task: BERT for text, ResNet for images.
2. Remove head	Drop the original output layer (trained for the original task like ImageNet 1000 classes).
3. Add new head	Add new output layers matching your task: 2 neurons for binary classification, etc.
4. Freeze base layers	Lock pre-trained weights so they don't change — only the new head trains initially.
5. Fine-tune	After head converges, optionally unfreeze some or all base layers and train end-to-end with a small LR.

Feature Extraction vs Fine-Tuning

Approach	Weights Updated When to Use
Feature Extraction	Only the new head. Base model frozen. Fast, needs little data. Use when task is similar to pre-training domain.
Partial Fine-Tuning	New head + last few layers of base. Most common approach. Balance speed and task adaptation.
Full Fine-Tuning	All layers updated. Needs more data and time. Use when task is very different from pre-training domain.
PEFT / LoRA	Only small adapter weights trained. Extremely efficient. Standard for fine-tuning LLMs.

Transfer Learning for Images — Keras Example

```
import tensorflow as tf
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, Model

# 1. Load pre-trained base (ImageNet weights, no top)
base = EfficientNetB0(weights='imagenet', include_top=False,
                      input_shape=(224, 224, 3))

# 2. Freeze base — only train new head initially
base.trainable = False

# 3. Build new model with custom head
inputs = tf.keras.Input(shape=(224, 224, 3))
x = base(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(5, activation='softmax')(x) # 5 custom classes

model = Model(inputs, outputs)
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 4. Train head only
model.fit(train_ds, epochs=10, validation_data=val_ds)

# 5. Unfreeze top layers for fine-tuning
```

```
base.trainable = True
for layer in base.layers[:-20]:
    layer.trainable = False

model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.fit(train_ds, epochs=20, validation_data=val_ds)
```

Transfer Learning for Text — BERT Example

```
from transformers import (BertTokenizer, TFBertForSequenceClassification,
                          create_optimizer)
import tensorflow as tf

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model      = TFBertForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2)

def tokenise(texts, max_len=128):
    return tokenizer(texts, truncation=True, padding='max_length',
                    max_length=max_len, return_tensors='tf')

optimizer, schedule = create_optimizer(
    init_lr=2e-5, num_warmup_steps=100, num_train_steps=1000)

model.compile(optimizer=optimizer,
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

model.fit(train_dataset, epochs=3, validation_data=val_dataset)
```

Lecture 4: How Transfer Learning is Performed in Industry

Industry Workflow

Phase	Details
1. Choose base model	Match domain (text->BERT/LLaMA, image->ResNet/ViT) and size (small for edge, large for accuracy)
2. Prepare dataset	Label data specific to your task. Even 500-5000 examples can be enough with transfer learning.
3. Feature extraction	Train only the head — fast validation that the approach works before committing to full fine-tuning
4. Fine-tuning	Unfreeze layers progressively. Use LR 10-100x smaller than pre-training LR to avoid catastrophic forgetting.
5. Evaluate	Domain-specific metrics: F1 for NER, BLEU for translation, mAP for detection, Rouge for summarisation
6. Deploy	Export to ONNX, TensorFlow Serving, TorchServe, or call via API (OpenAI, Anthropic, HuggingFace Inference)

Fine-Tuning LLMs in Industry — PEFT & LoRA

Full fine-tuning of a 7B parameter LLM requires 80GB+ of GPU VRAM. Parameter-Efficient Fine-Tuning (PEFT) techniques like LoRA make this possible on a single consumer GPU.

PEFT Method	How It Works Params Trained
LoRA (Low-Rank Adaptation)	Adds small low-rank matrices (A, B) to each attention layer. Only A and B train. ~0.1-1% of total params.
QLoRA	LoRA + quantise base model to 4-bit (NF4). Fine-tune 70B model on a single 48GB GPU.
Prefix Tuning	Prepend trainable 'prefix' tokens to input. Only prefix trains, full model frozen.
Prompt Tuning	Similar to prefix but only at the embedding layer — fewest trainable parameters.
Adapter Layers	Insert small trainable modules between frozen transformer layers.

```
# Fine-tuning LLaMA 3 with QLoRA (industry standard)
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, TaskType

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type='nf4',
    bnb_4bit_compute_dtype='float16'
)
base_model = AutoModelForCausalLM.from_pretrained(
    'meta-llama/Meta-Llama-3-8B',
    quantization_config=bnb_config,
    device_map='auto'
)

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,
    lora_alpha=32,
    target_modules=['q_proj', 'v_proj'],
    lora_dropout=0.05
)

model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()
# trainable: 0.0523% - only 4M of 8B params train!

from trl import SFTTrainer
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir='./llama3-finetuned',
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    fp16=True,
)

trainer = SFTTrainer(
    model=model,
    train_dataset=train_dataset,
    dataset_text_field='text',
    tokenizer=tokenizer,
    args=training_args,
    max_seq_length=512
)
```

PEFT Method	How It Works Params Trained
<code>trainer.train()</code>	

Catastrophic Forgetting

When you fine-tune a pre-trained model on new data, it can forget its original capabilities. This is called catastrophic forgetting.

Solution	How It Helps
Small learning rate	LR 10-100x lower than pre-training prevents large weight updates that erase prior knowledge
Gradual unfreezing	Unfreeze layers from top to bottom — let each layer adapt before exposing deeper ones
LoRA / PEFT	Base weights never change at all — only adapters train. Forgetting is impossible.
Elastic Weight Consolidation (EWC)	Penalise changes to weights that were important for the original task
Multi-task learning	Train on original task + new task simultaneously to retain both capabilities

Transfer Learning Decision Framework

Situation	Recommended Approach
Same domain, small dataset (< 1K)	Feature extraction only — freeze all, train just the head
Same domain, medium dataset (1K-10K)	Fine-tune last few layers + head with small LR
Different domain, large dataset (> 10K)	Full fine-tuning or train from scratch
LLM, any dataset size	LoRA or QLoRA — most practical and cost-effective
Production speed matters	Distil a fine-tuned large model into a smaller one (knowledge distillation)

```
# Inference with a fine-tuned LoRA model
from peft import PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer

base = AutoModelForCausalLM.from_pretrained('meta-llama/Meta-Llama-3-8B')
model = PeftModel.from_pretrained(base, './llama3-finetuned/checkpoint-best')
tokenizer = AutoTokenizer.from_pretrained('meta-llama/Meta-Llama-3-8B')

# Merge LoRA weights into base for faster inference
model = model.merge_and_unload()

prompt = 'Summarise the following medical report:'
inputs = tokenizer(prompt, return_tensors='pt').to('cuda')
outputs = model.generate(**inputs, max_new_tokens=200, temperature=0.7)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

End of Module 18 Notes — Generative AI Course